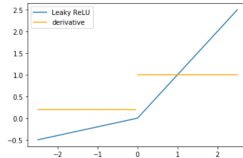


Advanced Artificial Intelligence and Machine Learning

DL Intro II



Learning goals

- Common activation functions for hidden units (sigmoid, tanh, ReLU and its variants) and the vanishing-gradient/saturation issues they address
- Output activations and their connection to loss functions via maximum likelihood
- Standard loss functions for regression (L2, L1) and classification (cross-entropy, softmax)
- Regularized risk minimization: norm penalties and weight decay
- Early stopping as an implicit regularizer, and its relationship to L2 regularization
- Ensemble methods, dropout, and data augmentation as further regularization techniques

Hidden activations

HIDDEN ACTIVATIONS

- Recall, hidden-layer activation functions make it possible for deep neural nets to learn complex non-linear functions.
- The design of hidden units is an extremely active area of research. It is usually not possible to predict in advance which activation will work best. Therefore, the design process often consists of trial and error.
- In the following, we will limit ourselves to the most popular activations - Sigmoidal activation and ReLU.
- It is possible for many other functions to perform as well as these standard ones. An overview of further activations can be found [▶ here](#).

SIGMOIDAL ACTIVATIONS

- Sigmoidal functions such as $\tanh$ and the *logistic sigmoid* bound the outputs to a certain range by "squashing" their inputs.

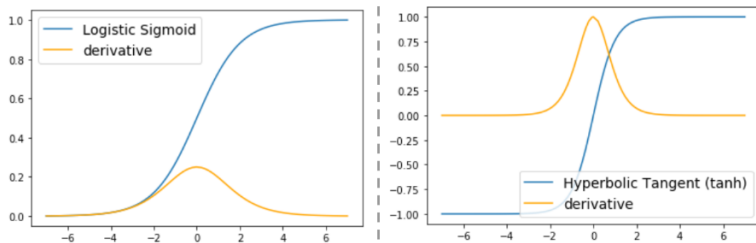


Figure: Sigmoidal activations (Savino & Tondolo, 2021)

- In each case, the function is only sensitive to its inputs in a small neighborhood around 0.
- Furthermore, the derivative is never greater than 1 and is close to zero across much of the domain.

SIGMOIDAL ACTIVATION FUNCTIONS

① Saturating Neurons:

- We know: $\sigma'(z_{in}) \rightarrow 0$ for $|z_{in}| \rightarrow \infty$.
- Neurons with sigmoidal activations "saturate" easily, that is, they stop being responsive when $|z_{in}| \gg 0$.

SIGMOIDAL ACTIVATION FUNCTIONS / 2

- ② **Vanishing Gradients:** Consider the vector of error signals $\delta^{(i)}$ in layer i

$$\delta^{(i)} = W^{(i+1)} \delta^{(i+1)} \odot \sigma' \left(\mathbf{z}_{in}^{(i)} \right), i \in \{1, \dots, O\}.$$

Each k -th component of the vector expresses how much the loss L changes when the input to the k -th neuron $z_{k,in}^{(i)}$ changes.

- We know: $\sigma'(z) < 1$ for all $z \in \mathbb{R}$.

SIGMOIDAL ACTIVATION FUNCTIONS / 3

→ In each step of the recursive formula above, the value will be multiplied by a value smaller than one

$$\begin{aligned}\delta^{(1)} &= W^{(2)}\delta^{(2)} \odot \sigma' \left(z_{in}^{(i)} \right) \\ &= W^{(2)} \left(W^{(3)}\delta^{(3)} \odot \sigma' \left(z_{in}^{(i)} \right) \right) \odot \sigma' \left(z_{in}^{(i)} \right) \\ &= \dots\end{aligned}$$

- When this occurs, earlier layers train *very* slowly (or not at all).

RECTIFIED LINEAR UNITS (RELU)

- The ReLU activation solves the vanishing gradient problem.

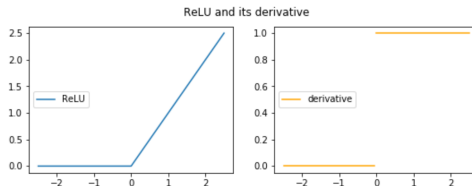


Figure: ReLU activation (Savino & Tondolo, 2021)

- In regions where the activation is positive, the derivative is 1.
- Derivatives do not vanish along paths containing "active" neurons.
- Note that the ReLU is not differentiable at 0 (Software implementations usually simply return 0).

RECTIFIED LINEAR UNITS (RELU)

- ReLU units can significantly speed up training compared to units with saturating activations.

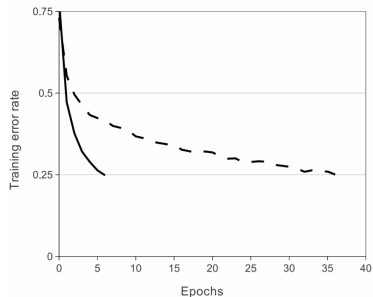


Figure: A four-layer convolutional neural network with ReLUs (solid line) reaches a 25% training error rate on the CIFAR-10 dataset six times faster than an equivalent network with tanh neurons (dashed line) (Krizhevsky et al., 2012).

RECTIFIED LINEAR UNITS (RELU)

- A downside of ReLU units is that when the input to the activation is negative, the derivative is zero (the "dying ReLU problem").

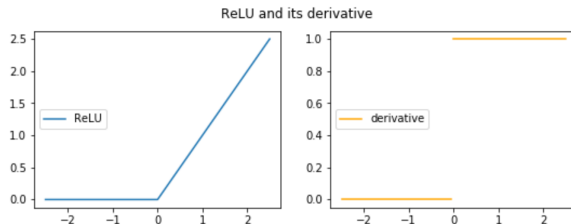


Figure: ReLU activation (Savino & Tondolo, 2021)

- When a ReLU unit "dies" (i.e. its activation is 0 for all datapoints), it kills the gradient flowing during backpropagation.
- Such units are never updated during training.

GENERALIZATIONS OF RELU

- There exist several generalizations of the ReLU activation that have non-zero derivatives throughout their domains.
- *Leaky ReLU*:

$$LReLU(v) = \begin{cases} v & v \geq 0 \\ \alpha v & v < 0 \end{cases}$$

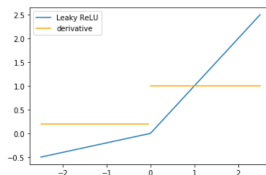


Figure: Leaky ReLU (Savino & Tondolo, 2021)

- Unlike the ReLU, when the input to the Leaky ReLU activation is negative, the derivative is α which is a small positive value.

GENERALIZATIONS OF RELU

- A variant of the Leaky ReLU is the *Parametric ReLU (PReLU)* which learns the α from the data through backpropagation.
- *Exponential Linear Unit (ELU)*:

$$ELU(v) = \begin{cases} v & v \geq 0 \\ \alpha(e^v - 1) & v < 0 \end{cases}$$

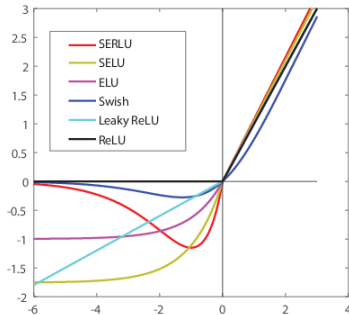
- *Scaled Exponential Linear Unit (SELU)*:

$$SELU(v) = \lambda \begin{cases} v & v \geq 0 \\ \alpha(e^v - 1) & v < 0 \end{cases}$$

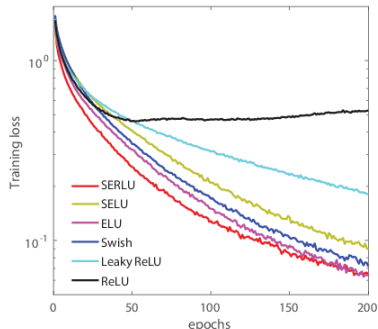
Note: In ELU and SELU, α and λ are hyperparameters that are set before training.

- These generalizations may perform as well as or better than the ReLU on some tasks.

GENERALIZATIONS OF RELU



(a): Different activation functions



(b): Performance on CIFAR10 without dropout

Figure: Visualization of different generalisations of ReLU (Zhang et al., 2018)

Output activations

OUTPUT ACTIVATIONS

- As we have seen previously, the role of the output activation is to get the final score on the same scale as the target.
- The output activations and the loss functions used to train neural networks can be viewed through the lens of maximum likelihood estimation (MLE).
- In general, the function $f(x | \theta)$ represented by the neural network defines the conditional $p(y | x, \theta)$ in a supervised learning task.
- Maximizing the likelihood is then equivalent to minimizing $-\log p(y | x, \theta)$.
- An output unit with the identity function as the activation can be used to represent the mean of a Gaussian distribution.
- For such a unit, training with mean-squared error is equivalent to maximizing the log-likelihood (ignoring issues with non-convexity).

EXCURSUS: MAXIMUM LIKELIHOOD ESTIMATION

- Given i.i.d. training data $(x^{(i)}, y^{(i)})_{i \in \{1, \dots, n\}}$ and a model defining $p(y \mid x, \theta)$, the **likelihood** is how probable the observed labels are under the model:

$$L(\theta) = \prod_{i=1}^n p(y^{(i)} \mid x^{(i)}, \theta)$$

- Maximum likelihood estimation (MLE)** picks the parameters that make the observed data most probable: $\hat{\theta} = \arg \max_{\theta} L(\theta)$.

EXCURSUS: MAXIMUM LIKELIHOOD ESTIMATION / 2

- In practice we use the **log-likelihood**: log turns the product into a numerically stable sum, and since log is monotonic, the maximizer is unchanged:

$$\log L(\boldsymbol{\theta}) = \sum_{i=1}^n \log p(y^{(i)} | x^{(i)}, \boldsymbol{\theta})$$

- Since we usually *minimize* losses, we equivalently minimize the **negative** log-likelihood:
 $\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \left(- \sum_{i=1}^n \log p(y^{(i)} | x^{(i)}, \boldsymbol{\theta}) \right).$
- This is the bridge used throughout: pick an output activation that defines a distribution, and the matching loss falls out as its negative log-likelihood.

OUTPUT ACTIVATIONS

- Similarly, sigmoid and softmax units can output the parameter(s) of a Bernoulli distribution and Categorical distribution, respectively.
- It is straightforward to show that when the label is one-hot encoded, training with the cross-entropy loss is equivalent to maximizing log-likelihood. [▶ Click here](#)
- Because these activations can saturate, an important advantage of maximizing log-likelihood is that the log undoes some of the exponentiation in the activation functions which is desirable when optimizing with gradient-based methods.
- For example, in the case of softmax, the loss is:

$$L(y, f(x)) = -f_{in,k} + \log \sum_{k'=1}^g \exp(f_{in,k'})$$

where k is the correct class. The first term, $-f_{in,k}$, does not saturate which means training can progress steadily even if the contribution of $f_{in,k}$ to the second term is negligible.

OUTPUT ACTIVATIONS

- A neural network can even be used to output the parameters of more complex distributions.
- A Mixture Density Network, for example, outputs the parameters of a Gaussian Mixture Model:

$$p(y|x) = \sum_{c=1}^m \phi^{(c)}(x) \mathcal{N}\left(y; \boldsymbol{\mu}^{(c)}(x), \boldsymbol{\Sigma}^{(c)}(x)\right)$$

where m the number of components in the mixture.

- In such a network, the output units are divided into groups.
- One group of output neurons with softmax activation represents the weights $(\phi^{(c)})$ of the mixture.
- Another group with the identity activation represents the means $(\boldsymbol{\mu}^{(c)})$ and yet another group with a non-negative activation function (such as ReLU or the exponential function) can represent the variances of the (typically) diagonal covariance matrices $\boldsymbol{\Sigma}^{(c)}$.

OUTPUT ACTIVATIONS

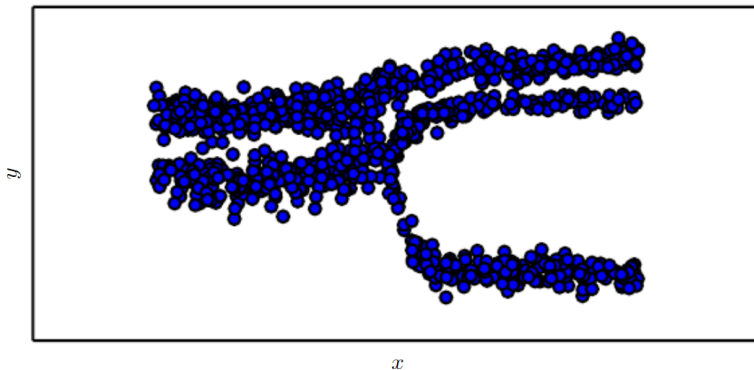


Figure: Samples drawn from a Mixture Density Network. The input x is sampled from a uniform distribution and y is sampled from $p(y | x, \theta)$ (Goodfellow et al., 2016).

REFERENCES

-  Savino, P., & Tondolo, F. (2021). Automated classification of civil structure defects based on convolutional neural network. *Frontiers of Structural and Civil Engineering*, 15(2), 305–317.
<https://doi.org/10.1007/s11709-021-0725-9>
-  Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
-  Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. In F. Pereira, C. J. Burges, L. Bottou, & K. Q. Weinberger (Eds.), *Advances in Neural Information Processing Systems* (Vol. 25). Curran Associates, Inc.
https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf
-  Zhang, G., & Li, H. (2018). *Effectiveness of Scaled Exponentially-Regularized Linear Units (SERLUs)*.

Loss Functions

WHAT IS A LOSS FUNCTION?

- A **loss function** $L(y, f(x))$ quantifies point-wise how we measure errors in predictions for a single x :

$$L : \mathcal{Y} \times \mathbb{R}^g \rightarrow \mathbb{R}.$$

- The **theoretical risk** of a candidate model f is the expected loss:

$$\mathcal{R}(f) := \mathbb{E}_{xy}[L(y, f(x))] = \int L(y, f(x)) \, d\mathbb{P}_{xy},$$

the average error we incur when using f on data from $\mathbb{P}_{xy}$. Since $\mathbb{P}_{xy}$ is unknown, we cannot compute this directly.

WHAT IS A LOSS FUNCTION? / 2

- Instead, given n i.i.d. training observations, we approximate it by the **empirical risk**:

$$\mathcal{R}_{\text{emp}} = \frac{1}{n} \sum_{i=1}^n L\left(y^{(i)}, f(x^{(i)} \mid \theta)\right).$$

- **Empirical risk minimization (ERM)** finds $\hat{\theta} = \arg \min_{\theta} \mathcal{R}_{\text{emp}}$, usually via numerical optimization.

REGRESSION: L2 AND L1 LOSS

- For regression, we typically use the L2 (squared-error) loss:

$$L(y, f(x)) = \frac{1}{2}(y - f(x))^2$$

- For a single neuron, this loss is convex; the global optimum can be found with gradient descent, or in this special case analytically via the normal equations.
- An alternative is the L1 (absolute-error) loss:

$$L(y, f(x)) = |y - f(x)|$$

- L2 loss is minimized (in expectation) by the conditional mean of y given x ; L1 loss is minimized by the conditional median. L1 is therefore more robust to outliers, but its loss surface is non-differentiable at 0.

REGRESSION: L2 AND L1 LOSS / 2

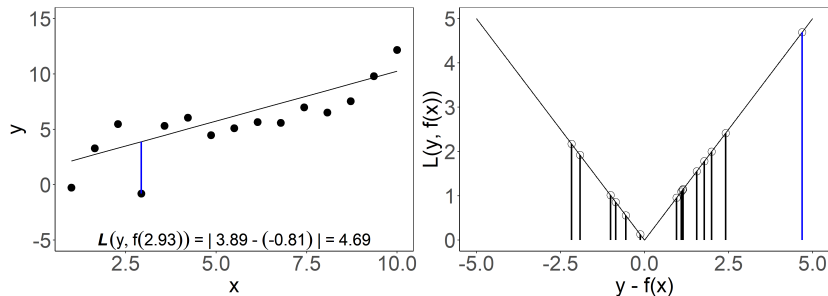


Figure: L1 loss: per-observation loss (left) and the loss curve $L(y, f(x))$ against the residual $y - f(x)$ (right). Source: SLDS LMU, I2ML.

REGRESSION: L2 AND L1 LOSS / 3

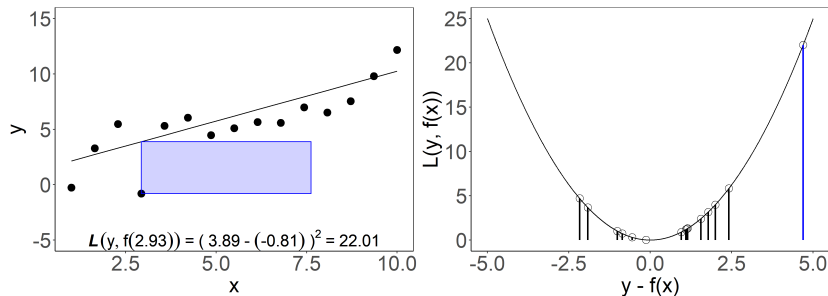


Figure: L2 loss: per-observation loss (left) and the loss curve $L(y, f(x))$ against the residual $y - f(x)$ (right). Source: SLDS LMU, I2ML.

BINARY CLASSIFICATION: CROSS-ENTROPY LOSS

- For binary classification, we typically apply the cross-entropy loss (also known as Bernoulli loss):

$$L(y, f(x)) = -(y \log f(x) + (1 - y) \log(1 - f(x)))$$

- A single neuron with logistic sigmoid activation trained with the Bernoulli loss yields the same result as logistic regression trained until convergence.

MULTI-CLASS CLASSIFICATION: SOFTMAX LOSS

- For g -class classification, the softmax loss is

$$L(y, f(x)) = - \sum_{k=1}^g [y = k] \log \left(\frac{\exp(f_{in,k})}{\sum_{k'=1}^g \exp(f_{in,k'})} \right)$$

- This is equivalent to the cross-entropy loss when the label vector $\mathbf{y}$ is one-hot coded.

MULTI-CLASS CLASSIFICATION:

SOFTMAX LOSS / 2

- A numerically more stable, equivalent form is

$$L(y, f(x)) = -f_{in,k} + \log \sum_{k'=1}^g \exp(f_{in,k'})$$

where k is the correct class: the first term does not saturate, so training progresses steadily even when $f_{in,k}$'s contribution to the second term is negligible.

LOSSES AS MAXIMUM LIKELIHOOD

- Output activations and loss functions can be viewed through the lens of maximum likelihood estimation (MLE): the network's output defines a conditional distribution $p(y \mid x, \theta)$, and maximizing the likelihood is equivalent to minimizing $-\log p(y \mid x, \theta)$.
- An identity output activation trained with mean-squared error is equivalent to maximizing the log-likelihood under a Gaussian distribution.
- Sigmoid/softmax output activations trained with cross-entropy loss are equivalent to maximizing the log-likelihood under a Bernoulli/Categorical distribution, respectively.

REFERENCES



SLDS LMU. *Introduction to Machine Learning (I2ML)*, chapter “Losses & Risk Minimization”. https://slds-lmu.github.io/i2ml/chapters/01_ml_basics/01-06-riskminimization/

Basic Regularization

REGULARIZATION

- Any technique that is designed to reduce the test error possibly at the expense of increased training error can be considered a form of regularization.
- Regularization is important in DL because NNs can have extremely high capacity (millions of parameters) and are thus prone to overfitting.

REVISION: REGULARIZED RISK MINIMIZATION

- The goal of regularized risk minimization is to penalize the complexity of the model to minimize the chances of overfitting.
- By adding a parameter norm penalty term $J(\theta)$ to the empirical risk $\mathcal{R}_{\text{emp}}(\theta)$ we obtain a regularized cost function:

$$\mathcal{R}_{\text{reg}}(\theta) = \mathcal{R}_{\text{emp}}(\theta) + \lambda J(\theta)$$

with hyperparameter $\lambda \in [0, \infty)$, that weights the penalty term, relative to the unconstrained objective function $\mathcal{R}_{\text{emp}}(\theta)$.

- Therefore, instead of pure **empirical risk minimization**, we add a penalty for complex (read: large) parameters θ .
- Declaring $\lambda = 0$ obviously results in no penalization.
- We can choose between different parameter norm penalties $J(\theta)$.
- In general, we do not penalize the bias.

L2-REGULARIZATION / WEIGHT DECAY

Let us optimize the L2-regularized risk of a model $f(x | \theta)$

$$\min_{\theta} \mathcal{R}_{\text{reg}}(\theta) = \min_{\theta} \mathcal{R}_{\text{emp}}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$$

by gradient descent. The gradient is

$$\nabla \mathcal{R}_{\text{reg}}(\theta) = \nabla \mathcal{R}_{\text{emp}}(\theta) + \lambda \theta.$$

We iteratively update θ by step size α times the negative gradient

$$\theta^{[t+1]} = \theta^{[t]} - \alpha \left(\nabla \mathcal{R}_{\text{emp}}(\theta) + \lambda \theta^{[t]} \right) = \theta^{[t]}(1 - \alpha\lambda) - \alpha \nabla \mathcal{R}_{\text{emp}}(\theta)$$

→ The term $\lambda \theta^{[t]}$ causes the parameter (**weight**) to **decay** in proportion to its size (which gives rise to the name).

EQUIVALENCE TO CONSTRAINED OPTIMIZATION

Norm penalties can be interpreted as imposing a constraint on the weights. One can show that

$$\arg \min_{\theta} \mathcal{R}_{\text{emp}}(\theta) + \lambda J(\theta)$$

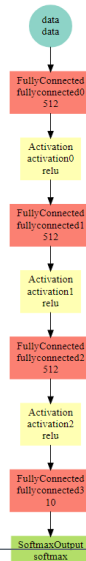
is equivalent to

$$\begin{aligned} & \arg \min_{\theta} \mathcal{R}_{\text{emp}}(\theta) \\ & \text{subject to } J(\theta) \leq k \end{aligned}$$

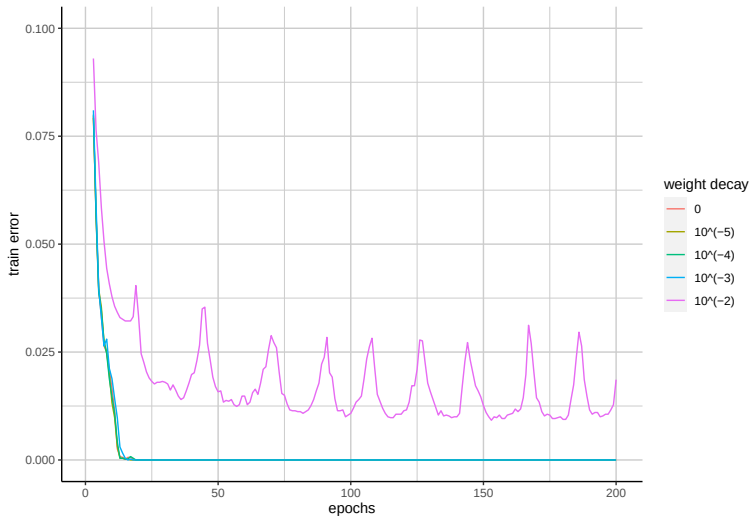
for some value k that depends on λ and $\mathcal{R}_{\text{emp}}(\theta)$ (Goodfellow et al., 2016)

EXAMPLE: WEIGHT DECAY

- We fit the huge neural network on the right side on a smaller fraction of MNIST (5000 train and 1000 test observations)
- Weight decay: $\lambda \in (10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, 0)$



EXAMPLE: WEIGHT DECAY / 2



A high weight decay of 10^{-2} leads to a high error on the training data.

TENSORFLOW PLAYGROUND

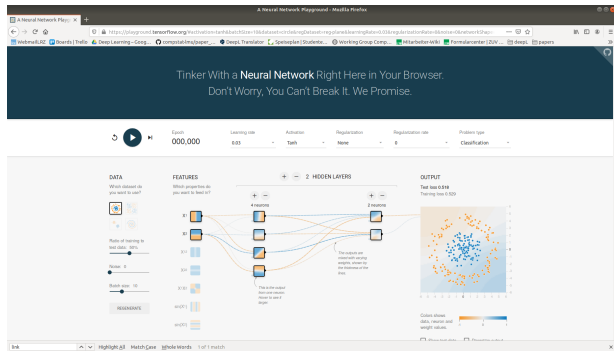


Figure: <https://playground.tensorflow.org/>, (Carter)

TENSORFLOW PLAYGROUND - EXERCISE

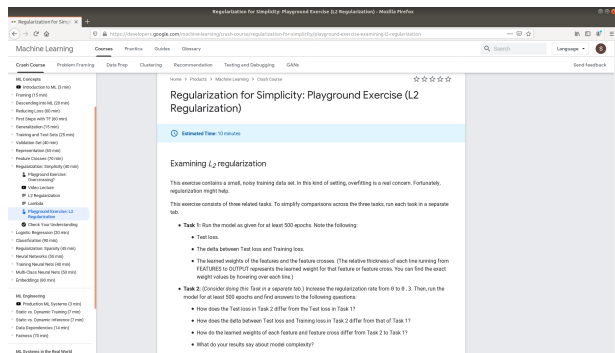


Figure: <https://developers.google.com/machine-learning/crash-course/regularization-for-simplicity/playground-exercise-examining-l2-regularization> (Google)

REFERENCES



Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.



MathWorks. (n.d.). *Regularization*. MATLAB & Simulink.
<https://de.mathworks.com/discovery/regularization.html>



Carter, D. S. and S. (n.d.). *Tensorflow - Neural Network Playground. A Neural Network Playground*. <https://playground.tensorflow.org>



Google. (n.d.). *Regularization for Simplicity: Playground Exercise (L2 Regularization)*. Google. <https://developers.google.com/machine-learning/crash-course/regularization-for-simplicity/playground-exercise-examining-l2-regularization?hl=de>

Early Stopping

EARLY STOPPING

- When training with an iterative optimizer such as SGD, it is commonly the case that, after a certain number of iterations, generalization error begins to increase even though training error continues to decrease.
- **Early stopping** refers to stopping the algorithm early before the generalization error increases.

EARLY STOPPING / 2

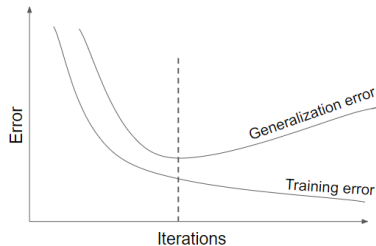


Figure: After a certain number of iterations, the algorithm begins to overfit.

Ensemble Methods

RECAP: ENSEMBLE METHODS

- Idea: Train **several models** separately, and **average their prediction** (i.e. perform **model averaging**).
- Intuition: This improves performance on test set, since different models will not make the same errors.
- Ensembles can be constructed in different ways, e.g.:
 - by combining completely different kind of models (using different learning algorithms and loss functions).
 - by **bagging**: train the same model on k datasets, constructed by sampling n samples from original dataset.
- Since training a neural network repeatedly on the same dataset results in different solutions (why?) it can even make sense to combine those.

RECAP: ENSEMBLE METHODS / 2

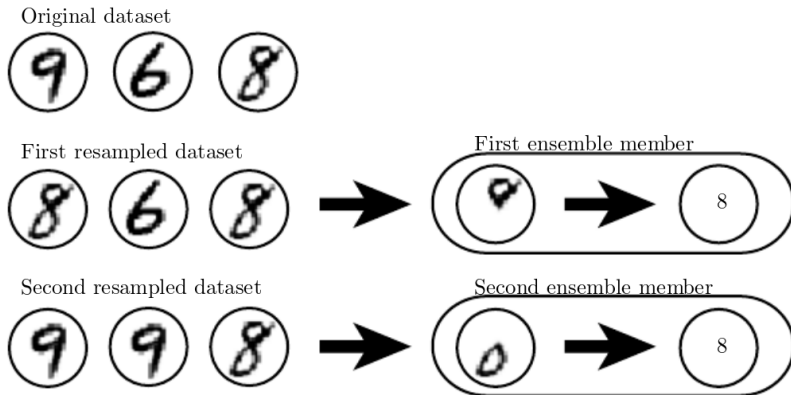


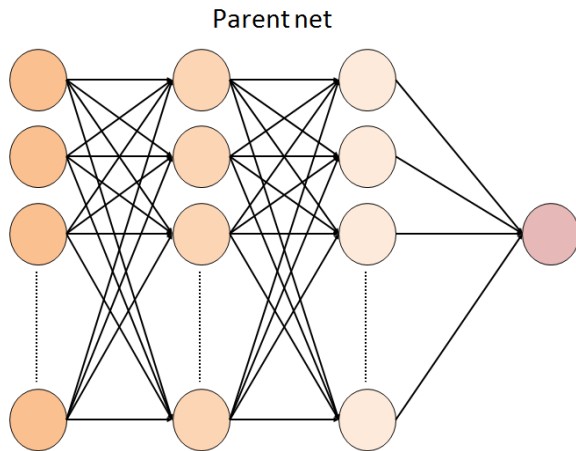
Figure: A cartoon description of bagging (Goodfellow et al., 2016)

Dropout

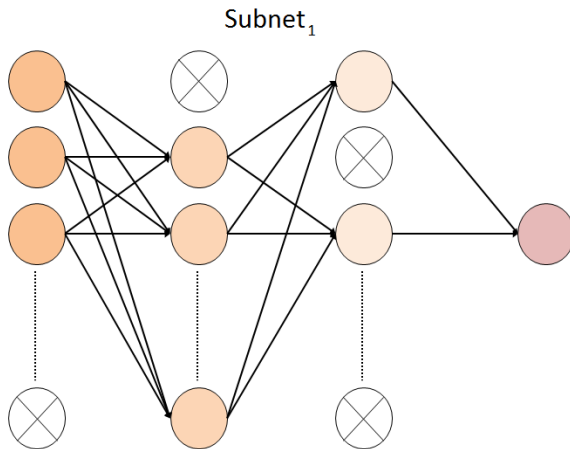
DROPOUT

- Idea: reduce overfitting in neural networks by preventing complex co-adaptations of neurons.
- Dropout can be seen as a form of "model averaging".
- Method: during training, random subsets of the neurons are removed from the network (they are "dropped out"). This is done by artificially setting the activations of those neurons to zero.
- Whether a given unit/neuron is dropped out or not is completely independent of the other units.
- If the network has N (input/hidden) units, applying dropout to these units can result in 2^N possible 'subnetworks'.
- Because these subnetworks are derived from the same 'parent' network, many of the weights are shared.

DROPOUT / 2



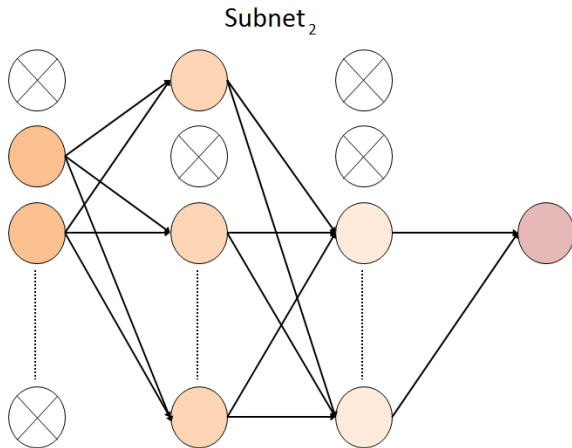
DROPOUT / 3



DROPOUT / 4

In each iteration, for each training example (in the forward pass), a different (random) subset of neurons are dropped out.

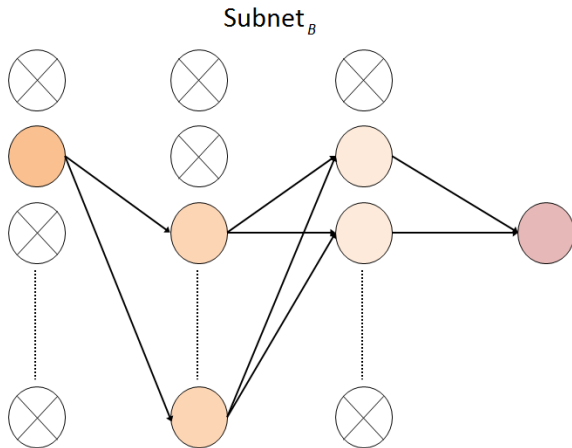
DROPOUT / 5



DROPOUT / 6

In each iteration, for each training example (in the forward pass), a different (random) subset of neurons are dropped out.

DROPOUT / 7



DROPOUT / 8

In each iteration, for each training example (in the forward pass), a different (random) subset of neurons are dropped out.

DROPOUT: ALGORITHM

- To train with dropout a minibatch-based learning algorithm such as stochastic gradient descent is used.
- For each training case in a minibatch, we randomly sample a binary vector/mask μ with one entry for each input or hidden unit in the network. The entries of μ are sampled independently from each other.
- The probability p of sampling a mask value of 0 (dropout) for one unit is a hyperparameter known as the 'dropout rate'.
- A typical value for the dropout rate is 0.2 for input units and 0.5 for hidden units.
- Each unit in the network is multiplied by the corresponding mask value resulting in a *subnet* $_{\mu}$.
- Forward propagation, backpropagation, and the learning update are run as usual.

DROPOUT: ALGORITHM / 2

Algorithm Training a (parent) neural network with dropout rate p

- 1: Define parent network and initialize weights
 - 2: **for** each minibatch: **do**
 - 3: **for** each training sample: **do**
 - 4: Draw mask μ using p
 - 5: Compute forward pass for $subnet_{\mu}$
 - 6: **end for**
 - 7: Update the weights of the (parent) network by performing a gradient descent step with weight decay
 - 8: **end for**
-

- The derivatives wrt. each parameter are averaged over the training cases in each mini-batch. Any training case which does not use a parameter contributes a gradient of zero for that parameter.

DROPOUT: WEIGHT SCALING

- The weights of the network will be larger than normal because of dropout. Therefore, to obtain a prediction at test time the weights must be first scaled by the chosen dropout rate.
- This means that if a unit (neuron) is retrained with probability p during training, the weight at test time of that unit is multiplied by p .

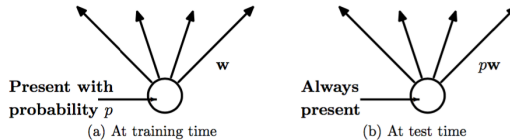


Figure: Training vs. Testing (Srivastava et. al., 2014)

DROPOUT: WEIGHT SCALING / 2

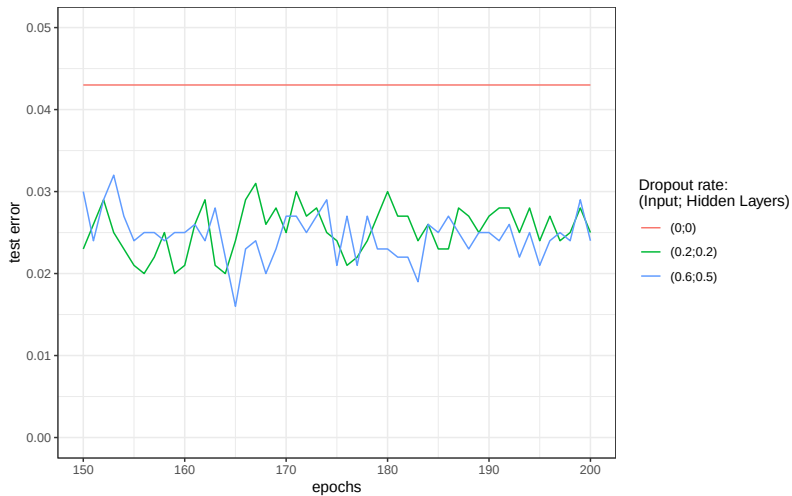
- Weight scaling ensures that the expected total input to a neuron/unit at test time is roughly the same as the expected total input to that unit at train time, even though many of the units at train time were missing on average
- Rescaling of the weights can also be performed at training time instead, after each weight update at the end of the mini-batch. This is sometimes called 'inverse dropout'. Keras and PyTorch deep learning libraries implement dropout in this way.

DROPOUT: EXAMPLE

- To demonstrate how dropout can easily improve generalization we compute neural networks with the structure showed on the right.
- Each neural network we fit has different dropout probabilities, a tuple where one probability is for the input layer and one is for the hidden layers. We consider the tuples $(0; 0)$, $(0.2; 0.2)$ and $(0.6; 0.5)$.



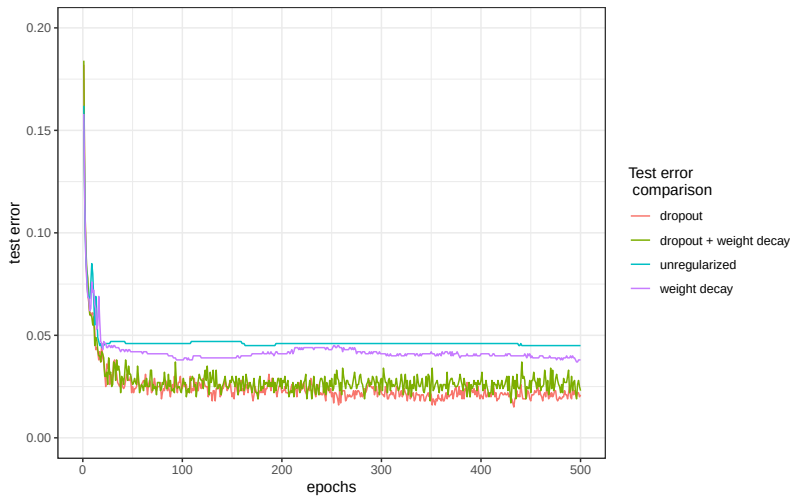
DROPOUT: EXAMPLE / 2



DROPOUT: EXAMPLE / 3

Dropout rate of 0 (no dropouts) leads to higher test error than dropping some units out.

DROPOUT, WEIGHT DECAY OR BOTH?



DROPOUT, WEIGHT DECAY OR BOTH? / 2

Here, dropout leads to a smaller test error than using no regularization or solely weight decay.

Dataset Augmentation

DATASET AUGMENTATION

- Problem: low generalization because high ratio of

$$\frac{\text{complexity of the model}}{\# \text{train data}}$$

- Idea: artificially increase the train data.
 - Limited data supply → create “fake data”!
- Increase variation in inputs **without** changing the labels.
- Application:
 - Image and Object recognition (rotation, scaling, pixel translation, flipping, noise injection, vignetting, color casting, lens distortion, injection of random negatives)
 - Speech recognition (speed augmentation, vocal tract perturbation)

DATASET AUGMENTATION / 2



(a) Original



(b) Color



(c) Rotate



(d) Horizontal Stretch

DATASET AUGMENTATION / 3

Figure: Example for data set augmentation (Wu et al., 2015)

DATASET AUGMENTATION / 4

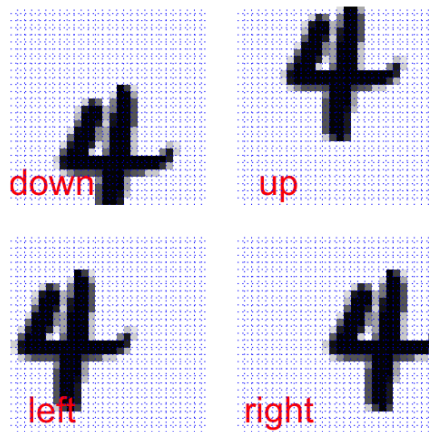


Figure: Example for data set augmentation (Wu et al., 2015)

DATASET AUGMENTATION / 5

⇒ careful when rotating digits (6 will become 9 and vice versa)!

REFERENCES

-  Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
-  Hinton, G. E., Srivastava, N., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. R. (2012). Improving neural networks by preventing co-adaptation of feature detectors.
-  Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15(56), 1929–1958.
<http://jmlr.org/papers/v15/srivastava14a.html>
-  Wu, R., Yan, S., Shan, Y., Dang, Q., & Sun, G. (2015). Deep Image: Scaling up Image Recognition.

LICENSE & ATTRIBUTION

- Activations and Regularization sections adapted from **Introduction to Deep Learning (I2DL)**, Prof. Dr. Bernd Bischl, Statistical Learning and Data Science (SLDS), LMU Munich. https://github.com/slds-lmu/lecture_i2dl — licensed **CC BY 4.0**:
<https://creativecommons.org/licenses/by/4.0/>
- Loss Functions section assembled from **Introduction to Machine Learning (I2ML)**, SLDS, LMU Munich. <https://github.com/slds-lmu/i2ml> — licensed **MIT**.
- Content has been merged, reordered, and consolidated from the original sources into this single session deck; not used verbatim.